

MATPLOTLIB

import numpy as np

import pandas as pd

Matplotlib

Matplotlib is one of the most popular data visualization libraries in Python. It allows us to create a wide range of plots — from simple line charts and bar graphs to more advanced visualizations like histograms, scatter plots, and pie charts. Think of it as the tool that helps us see our data instead of just reading numbers.

It was originally developed to bring MATLAB-like plotting capabilities into Python, and today it serves as the foundation for many other visualization libraries such as Seaborn and Pandas plotting.

With Matplotlib, we can:

Quickly plot data for exploration.

Customize every aspect of a graph — titles, labels, colors, markers, legends, and styles.

Create high-quality visualizations for reports, research, and presentations.

Work seamlessly with NumPy and Pandas, making it perfect for data science and machine learning.

Matplotlib has following functionalities

```
# * Plot
# * Subplot
# * figure
# * subplots
# * pie
# * hist
# * scatter
```

```
import matplotlib.pyplot as plt
```

```
# Syntax: plt.plot(x,y)
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y)
```

```
# title
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y)
```

```
plt.title('Sakshi')
```

```
# show ---> To remove the unwanted heading text
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y)
```

```
plt.title('Sakshi')
```

```
plt.show() # It should be mentioned at the last
```

```
# x-label and y-label ---> for labelling the axis
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y)
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.show()
```

```
# You can plot multiple things also in a single plot
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y)
```

```
plt.plot(x, x)
```

```
plt.plot(y, x)
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.show()
```

Inorder to point or label the particular plots we have to use label in every plot and to display that use legend

Sample data

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]

Create a line plot

plt.plot(x, y, label='me')

plt.plot(x, x, label='you')

plt.plot(y, x, label='others')

plt.title('Sakshi')

plt.xlabel('x-axis')

plt.ylabel('y-axis')

plt.legend()

plt.show()

annotate ---> It is used to mark the points

Syntax:

for i in range(len(x)):

plt.annotate(f'{x[i],y[i]}', (x[i],y[i]))

Sample data

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]

```
# Create a line plot
plt.plot(x, y, label='me')

plt.title('Sakshi')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.legend()

for i in range(len(x)):
    plt.annotate(f'{x[i],y[i]}', (x[i], y[i]))

plt.show()
```

Marker To mark the points

Matplotlib Markers

'o' Circle

'.' Point

',' Pixel

'v' Triangle Down

'^' Triangle Up

'<' Triangle Left

'>' Triangle Right

'1' Tri Down (Thin)

'2' Tri Up (Thin)

'3' Tri Left (Thin)

'4' Tri Right (Thin)

's' Square

'p' Pentagon

'*' Star

'h' Hexagon1

'H' Hexagon2

'+' Plus

'x' Cross

'D' Diamond

'd' Thin Diamond

'|' Vertical Line

'_' Horizontal Line

Matplotlib Line Styles

'-'

'--'

'-.'

':'

Solid line

Dashed line

Dash-dot line

Dotted line

'None'

No line

Sample data

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y ,label='me',marker='o')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend()
```

```
for i in range(len(x)):
```

```
    plt.annotate(f'{x[i],y[i]}',(x[i],y[i]))
```

```
plt.show()
```

```
# color ---> It is used to change the color of the plot
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y ,label='me',marker='o',color='purple')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```



```
plt.legend()

# for i in range(len(x)):

# plt.annotate(f'{x[i],y[i]}',(x[i],y[i]))

plt.show()
```

Markerfacecolor ---> It is used to give color for the marker

```
# Sample data

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]
```

```
# Create a line plot

plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red')
```

```
plt.title('Sakshi')

plt.xlabel('x-axis')

plt.ylabel('y-axis')

plt.legend()

# for i in range(len(x)):

# plt.annotate(f'{x[i],y[i]}',(x[i],y[i]))

plt.show()
```

Markeredgcolor ---> It is used to add color for the marker border

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',  
markeredgecolor = 'black')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend()
```

```
# for i in range(len(x)):
```

```
# plt.annotate(f'{x[i],y[i]}', (x[i],y[i]))
```

```
plt.show()
```

```
# Markersize ---> It is used to change the size of the marker
```

```
# Linewidth ---> It is used to change the width of line
```

```
# LineStyle ---> It is used style for the line
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',  
markeredgecolor = 'black', markersize=8)
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend()
```

```
# for i in range(len(x)):
```

```
# plt.annotate(f'{x[i],y[i]}', (x[i], y[i]))
```

```
plt.show()
```

```
# Linewidth ---> It is used to change the width of line
```

```
# LineStyle ---> It is used style for the line
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',  
markeredgecolor = 'black', markersize=8, linewidth=3)
```

```
plt.title('Sakshi')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.legend()
# for i in range(len(x)):
# plt.annotate(f'{x[i],y[i]}',(x[i],y[i]))
plt.show()
```

LineStyle ---> It is used style for the line

Sample data

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

Create a line plot

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend()
```

```
# for i in range(len(x)):
```

```
# plt.annotate(f'{x[i],y[i]}',(x[i],y[i]))
```

```
plt.show()
```

```
# loc ---> It determines where the legend should be placed
```

```
# loc = 'best','upper right','upper left','lower left','lower right','right','center left','center  
right','low center','upper center','center'
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',  
markeredgecolor = 'black', markersize=8, linewidth=3, linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend(loc='upper center')
```

```
# for i in range(len(x)):
```

```
# plt.annotate(f'{x[i],y[i]}',(x[i],y[i]))
```

```
plt.show()
```

```
# title() ---> It is going to give title for legend
```

```
# Sample data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Create a line plot
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')

plt.title('Sakshi')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.legend(loc='upper center', title='Hello')
# for i in range(len(x)):
# plt.annotate(f'{x[i],y[i]}', (x[i],y[i]))
plt.show()
```

facecolor ---> It is going to give color for legend

```
# Sample data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Create a line plot
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')
```

```
plt.title('Sakshi')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.legend(loc='upper center',title='Hello',facecolor='grey')
# for i in range(len(x)):
# plt.annotate(f'{x[i],y[i]}',(x[i],y[i]))
plt.show()
```

fontsize ---> It is used to give font size for the legend

Sample data

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]

Create a line plot

```
plt.plot(x, y ,label='me',marker='o',color='purple', markerfacecolor = 'red',
markeredgecolor = 'black',markersize=8, linewidth=3,linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend(loc='upper center',title='Hello',facecolor='grey',fontsize = 10)
```

```
# for i in range(len(x)):
```

```
# plt.annotate(f'{x[i],y[i]}',(x[i],y[i]))
```

```
plt.show()
```

title_fontsize ---> It is used to give font size for the title of legend

Sample data

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]

Create a line plot

plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',
markeredgecolor = 'black', markersize=8, linewidth=3, linestyle=':')

plt.title('Sakshi')

plt.xlabel('x-axis')

plt.ylabel('y-axis')

plt.legend(loc='upper center', title='Hello', facecolor='grey', fontsize = 10, title_fontsize=15)

for i in range(len(x)):

plt.annotate(f'{x[i],y[i]}', (x[i], y[i]))

plt.show()

shadow ---> It is used to give shadows

Sample data

x = [1, 2, 3, 4, 5]


```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',  
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend(loc='upper center', title='Hello', facecolor='grey', fontsize =  
10, title_fontsize=15, shadow = True)
```

```
# for i in range(len(x)):
```

```
# plt.annotate(f'{x[i],y[i]}', (x[i],y[i]))
```

```
plt.show()
```

```
# ALIGNMENT ---> Aligning the plots
```

```
# ha ---> It is used to give horizontal alignment
```

```
# ha = 'center','left','right'
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',  
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend(loc='upper center', title='Hello', facecolor='grey', fontsize =  
10, title_fontsize=15, shadow = True)
```

```
for i in range(len(x)):
```

```
    plt.annotate(f'{x[i],y[i]}', (x[i], y[i]), ha='right')
```

```
plt.show()
```

ALIGNMENT ---> Aligning the plots

color ---> It is used to give color to the annotate

Sample data

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

Create a line plot

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',  
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')

plt.legend(loc='upper center',title='Hello',facecolor='grey',fontsize =
10,title_fontsize=15,shadow = True)

for i in range(len(x)):

    plt.annotate(f'{x[i],y[i]}',(x[i],y[i]), ha='right', va = 'bottom')

plt.show()
```

ALIGNMENT ---> Aligning the plots

color ---> It is used to give color to the annotate

Sample data

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

Create a line plot

```
plt.plot(x, y ,label='me',marker='o',color='purple', markerfacecolor = 'red',
markedgecolor = 'black',markersize=8, linewidth=3,linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend(loc='upper center',title='Hello',facecolor='grey',fontsize =
10,title_fontsize=15,shadow = True)
```

```
for i in range(len(x)):
```

```
    plt.annotate(f'{x[i],y[i]}',(x[i],y[i]), ha='right', va = 'bottom',color='green')
```

```
plt.show()
```

```
# Based on placing of the x-axis and y-axis it has following features
```

```
# xticks ---> It is used to rotate the x axis values
```

```
# Syntax: plt.xticks(rotation=angle)
```

```
# yticks ---> It is used to rotate the y axis values
```

```
# Syntax: plt.yticks(rotation=angle)
```

```
# axhline ---> It is used to draw a horizontal line
```

```
# Syntax: plt.axhline('value')
```

```
# axvline ---> It is used to draw a vertical line
```

```
# Syntax: plt.axvline('value')
```

```
# Sample data
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot

plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')

plt.title('Sakshi')

plt.xlabel('x-axis')

plt.ylabel('y-axis')

plt.legend(loc='upper center', title='Hello', facecolor='grey', fontsize =
10, title_fontsize=15, shadow = True)

for i in range(len(x)):

    plt.annotate(f'{x[i],y[i]}', (x[i],y[i]), ha='right', va = 'bottom', color='green')

plt.xticks(rotation=45)

plt.show()
```

Based on placing of the x-axis and y-axis it has following features

xticks ---> It is used to rotate the x axis values

Syntax: plt.xticks(rotation=angle)

Sample data

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]

```
# Create a line plot

plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')

plt.title('Sakshi')

plt.xlabel('x-axis')

plt.ylabel('y-axis')

plt.legend(loc='upper center', title='Hello', facecolor='grey', fontsize =
10, title_fontsize=15, shadow = True)

for i in range(len(x)):

    plt.annotate(f'{x[i],y[i]}', (x[i],y[i]), ha='right', va = 'bottom', color='green')

plt.xticks(rotation=45)

plt.show()
```

Based on placing of the x-axis and y-axis it has following features

yticks ---> It is used to rotate the y axis values

Syntax: plt.yticks(rotation=angle)

Sample data

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]

Create a line plot

```
plt.plot(x, y ,label='me',marker='o',color='purple', markerfacecolor = 'red',
markedgecolor = 'black',markersize=8, linewidth=3,linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend(loc='upper center',title='Hello',facecolor='grey',fontsize =
10,title_fontsize=15,shadow = True)
```

```
for i in range(len(x)):
```

```
    plt.annotate(f'{x[i],y[i]}',(x[i],y[i]), ha='right', va = 'bottom',color='green')
```

```
plt.xticks(rotation=45)
```

```
plt.yticks(rotation=90)
```

```
plt.show()
```

Based on placing of the x-axis and y-axis it has following features

axhline ---> It is used to draw a horizontal line

Syntax: plt.axhline('value')

Sample data

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot

plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',
markedgecolor = 'black', markersize=8, linewidth=3, linestyle=':')

plt.title('Sakshi')

plt.xlabel('x-axis')

plt.ylabel('y-axis')

plt.legend(loc='upper center', title='Hello', facecolor='grey', fontsize =
10, title_fontsize=15, shadow = True)

for i in range(len(x)):

    plt.annotate(f'{x[i],y[i]}', (x[i],y[i]), ha='right', va = 'bottom', color='green')

plt.xticks(rotation=45)

plt.yticks(rotation=90)

plt.axhline('bye')

plt.show()
```

Based on placing of the x-axis and y-axis it has following features

axvline ---> It is used to draw a vertical line

Syntax: plt.axvline('value')

Sample data


```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 6, 8, 10]
```

```
# Create a line plot
```

```
plt.plot(x, y, label='me', marker='o', color='purple', markerfacecolor = 'red',  
markeredgecolor = 'black', markersize=8, linewidth=3, linestyle=':')
```

```
plt.title('Sakshi')
```

```
plt.xlabel('x-axis')
```

```
plt.ylabel('y-axis')
```

```
plt.legend(loc='upper center', title='Hello', facecolor='grey', fontsize =  
10, title_fontsize=15, shadow = True)
```

```
for i in range(len(x)):
```

```
    plt.annotate(f'{x[i],y[i]}', (x[i],y[i]), ha='right', va = 'bottom', color='green')
```

```
plt.xticks(rotation=45)
```

```
plt.yticks(rotation=90)
```

```
plt.axhline(6)
```

```
plt.axvline(4)
```

```
plt.show()
```

```
# Let us practice through the datasets
```

```
df=pd.read_csv('https://raw.githubusercontent.com/Taj1920/Data-Analysis-  
Datasets/main/Datasets-Practice/titanic.csv',encoding='ISO-8859-1')
```

```
df
```

```
df1 = df.pivot_table(values='fare',  
                      index = 'pclass',  
                      aggfunc = 'mean'  
                      )
```

```
df1.reset_index(inplace=True)  
df1
```

```
x = df1['pclass']  
y = df1['fare']
```

```
plt.plot(x,y)  
plt.show()
```

```
#  Notes on Pivot Tables and Visualization with Matplotlib
```

```
---
```

```
### 1. Reading the Dataset
```

```
```python
```

```
df1 = pd.read_csv('/content/sales_dataset.csv')
```

df1

`,`,`

\* `pd.read\_csv()` → Reads the CSV file into a **Pandas DataFrame**.

\* `df1` → Contains sales data with columns like `Region`, `Product`, `TotalSales`, etc.

\* Displaying `df1` shows the raw dataset.

---

### ### 2. Pivot Table: Total Sales by Region

```python

```
a = df1.pivot_table(values='TotalSales', index='Region', aggfunc='sum')
```

```
a.reset_index(inplace=True)
```

a

`,`,`

* `pivot\_table()` → Summarizes data.

* `values='TotalSales'` → We want to aggregate sales.

* `index='Region'` → Groups data by region.

* `aggfunc='sum'` → Adds up sales for each region.

* `reset\_index(inplace=True)` → Converts the index (`Region`) back into a column for easier plotting.

* `a` → A DataFrame with total sales for each region.

3. Line Plot: Sales by Region

```
```python
x = a['Region']
y = a['TotalSales']

plt.plot(x, y)
plt.show()
```
```

- * `x = a['Region']` → X-axis contains region names.
- * `y = a['TotalSales']` → Y-axis contains total sales values.
- * `plt.plot(x,y)` → Creates a line plot of sales by region.
- * `plt.show()` → Displays the plot.

 ****Insight:**** Helps identify which regions contribute the most/least to sales.

4. Pivot Table: Total Sales by Product

```
```python
a = df1.pivot_table(values='TotalSales', index='Product', aggfunc='sum')
a.reset_index(inplace=True)
```
```

- * Groups sales data by `Product` instead of `Region`.
- * Shows total sales for each product across all regions.

5. Line Plot: Sales by Product

```
```python
x = a['Product']
y = a['TotalSales']
```

```
plt.plot(x,y)
plt.show()
```
```

- * Plots sales trends for each product.
- * Useful for comparing which products generate more revenue.

 ****Insight:**** Without rotation, product names might overlap if they're long.

6. Improved Plot: Rotated X-axis Labels

```
```python
x = a['Product']
y = a['TotalSales']

plt.plot(x,y)
plt.xticks(rotation=90)
```

```
plt.show()
```

```
` ``
```

\* Same plot as before, but `plt.xticks(rotation=90)` rotates product names vertically.

\* Prevents overlapping labels and improves readability.



**\*\*Insight:\*\*** Now you can clearly compare all products without clutter.

```

```



**\*\*Summary\*\*:**

\* You used **\*\*pivot tables\*\*** to group sales data by **\*\*Region\*\*** and **\*\*Product\*\***.

\* You plotted the results using **\*\*line plots\*\*** to visualize sales trends.

\* Rotation of X-axis labels makes categorical plots (like products) easier to read.

```

```

## # SUBPLOTS

### ### 1. Introduction to Subplots

```
` `` python
```

```
Subplots ---> We can plot multiple plots into multiple grids
```

```
Syntax: plt.subplot(no_of_rows, no_of_columns, positions)
```

```
` ``
```

\* \*\*`plt.subplot()`\*\* allows plotting multiple graphs \*\*in the same figure\*\* in a grid layout.

\* \*\*Parameters\*\*:

\* `no\_of\_rows` → Number of rows in the grid

\* `no\_of\_columns` → Number of columns in the grid

\* `position` → The cell number where the plot will appear (counted left-to-right, top-to-bottom)

---

### ### 2. Example: Creating Multiple Subplots

```
```python
```

```
plt.subplot(4,4,1)
```

```
plt.subplot(4,4,4)
```

```
plt.subplot(4,4,13)
```

```
plt.subplot(4,4,16)
```

```
```
```

\* Creates a \*\*4x4 grid\*\* (16 cells) of subplots.

\* Each `plt.subplot(4,4,position)` targets a specific subplot cell:

\* `1` → top-left

\* `4` → top-right

\* `13` → bottom-left

\* `16` → bottom-right

\* \*\*Note:\*\* Currently, no plots are drawn yet; these just define subplot positions.

---

### ### 3. Using `tight\_layout` for Spacing

```
```python
```

```
# tight layout ---> To give space between the subplots
```

```
plt.subplot(4,4,1)
```

```
plt.subplot(4,4,15)
```

```
plt.subplot(4,4,16)
```

```
plt.tight_layout()
```

```
```
```

\* `plt.tight\_layout()` automatically adjusts **spacing between subplots** to prevent overlapping of labels, titles, and axes.

\* Useful when plotting multiple charts in the same figure.

---

### ### 4. Combining Plots in Different Grid Sizes

```
```python
```

```
plt.subplot(2,1,1)
```

```
plt.plot(x,y)
```

```
plt.xticks(rotation=75)
```

```
plt.subplot(2,4,8)
```

```
plt.tight_layout()
```


`, `

* `plt.subplot(2,1,1)` → Creates a **2-row, 1-column** grid and selects the first row.

* `plt.plot(x,y)` → Plots the line graph in this subplot.

* `plt.xticks(rotation=75)` → Rotates X-axis labels for better readability.

* `plt.subplot(2,4,8)` → Creates a **2-row, 4-column** grid and targets the 8th subplot.

* `plt.tight\_layout()` → Ensures all subplots in the figure have **enough spacing**.

To make sure the subplots are visible write this

```
df1 = pd.read_csv('/content/sales_dataset.csv')
```

```
a = df1.pivot_table(values='TotalSales', index='Product', aggfunc='sum')
```

```
a.reset_index(inplace=True)
```

```
x = a['Product']
```

```
y = a['TotalSales']
```

```
plt.subplot(2,1,1)
```

```
plt.plot(x,y)
```

```
plt.subplot(2,1,2)
```

```
plt.title('hi')
```

```
plt.xticks(rotation=75)
```

```
plt.tight_layout()
```

🎯 Training Notes – Matplotlib Figure and Axes

1. **Figure Basics**

* A **Figure** is like a **big canvas** on which you draw your plots.

* Think of it like a blank sheet of paper where you can add smaller drawing boxes (called **Axes**).

Syntax:

```
```python
var = plt.figure()
```
```

Example:

```
```python
a = plt.figure()
```
```

👉 This creates an empty figure.

2. **Axes** (the small boxes inside a figure)

- * An **Axes** is the actual area where the data/graph is drawn.

- * You can place multiple Axes inside a single Figure at different positions.

- * The values `[x, y, width, height]` define where the Axes sits inside the Figure.

- * `x, y` → bottom-left corner position

- * `width, height` → size of the box

Syntax:

```
```python
var1 = var.add_axes([x, y, width, height])
```
```

Example:

```
```python
a = plt.figure()
b = a.add_axes([0, 0, 1, 1]) # big box
c = a.add_axes([0.5, 0.5, 0.4, 0.4]) # small box inside big
```
```

👉 `c` sits inside `b`.

3. ****Adjusting Axes Position****

* If you want to ****move the small box down or reduce its size****, change the values.

Example:

```
```python
a = plt.figure()
b = a.add_axes([0, 0, 1, 1])
c = a.add_axes([0.3, 0.2, 0.4, 0.4])
```
```

👉 Here, `c` is smaller and lower because `y=0.2`.

4. ****Figure Face Color****

* `facecolor` changes the ****background border color**** of the entire figure.

Example:

```
```python
a = plt.figure(facecolor='green')
b = a.add_axes([0, 0, 1, 1])
c = a.add_axes([0.5, 0.5, 0.4, 0.4])
```
```

👉 The figure border becomes green.

5. **Figure Size**

* Use `figsize=(width, height)` to control figure size (in **inches**).

Example:

```
```python
a = plt.figure(facecolor='green', figsize=(10,5))
```
```

👉 This creates a **wider figure** (10 inches wide, 5 inches tall).

6. **Adding Plots to Axes**

* Once you create Axes, you can **plot inside them**.

* Example:

```
```python
a = plt.figure(facecolor='green', figsize=(10,5))

d = a.add_axes([0, 0, 2, 1])

d.plot(x, y)
```

`, `

👉 Here, data `(x, y)` is plotted inside Axes `d`.

---

## # 📄 Summary Notes (Quick Revision)

\* **Figure** = canvas (whole page)

\* **Axes** = box inside the canvas where plots are drawn

\* **add\_axes([x, y, width, height])** → controls position & size of Axes

\* **facecolor** → sets figure background color

\* **figsize=(w, h)** → sets figure size in inches

\* You can create **multiple Axes** inside one Figure and plot separately

---

```
df1 = pd.read_csv('/content/sales_dataset.csv')
```

```
a = df1.pivot_table(values='TotalSales', index='Product', aggfunc='sum')
```

```
a.reset_index(inplace=True)
```

```
x = a['Product']
```

```
y = a['TotalSales']
```

```

a = plt.figure() # Create a new Figure object (the canvas for all plots)

b = a.add_axes([0,0,1,1]) # Add first Axes (main plot) covering entire figure [x0, y0,
width, height]

plt.xticks(rotation=90) # Rotate x-axis ticks of the current active axes by 90°

c = a.add_axes([0.5,0.5,0.4,0.4]) # Add second Axes (an inset/mini plot) inside the first
one

b.plot(x,y) # Plot (x,y) on the main Axes 'b'

b.set_title('Sakshi') # Title of the main plot

b.set_xlabel('Me') # X-axis label of the main plot

plt.xticks(rotation=90) # Again rotate x-ticks of the currently active axes

c.plot(x,y) # Plot (x,y) on the inset Axes 'c'

plt.show() # Display everything

```

**##** ♦ `subplot` vs `subplots` in Matplotlib

**#** ---

**###** 1 `plt.subplot()`

**# \*** Used to create **\*\*one subplot at a time\*\*** inside a grid.

**# \*** You define **\*\*rows, columns, and index position\*\*** for each subplot.

# \* Returns a **single Axes object**.

# **Syntax:**

# ```python

# plt.subplot(nrows, ncols, index)

# ```

# **Example:**

# ```python

# plt.subplot(2,2,1) # 2x2 grid → position 1

# plt.plot(x,y)

# plt.subplot(2,2,4) # 2x2 grid → position 4

# plt.plot(x,y)

# ```

# ---

# ## 2 `plt.subplots()`

# \* Used to create **multiple subplots at once** in a single figure.

# \* Returns both the **Figure object** and an **array of Axes objects**.

# \* Recommended for modern code since it's cleaner and more flexible.

# **Syntax:**

# fig, axes = plt.subplots(nrows=value, ncols=value)



```
Example:
```

```
```python
```

```
fig, axes = plt.subplots(2,2) # Create 2x2 grid
```

```
axes[0,0].plot(x,y) # Top-left
```

```
axes[1,1].plot(x,y) # Bottom-right
```

```
```
```

```

```

```
🗝️ Key Differences Between `subplot` and `subplots`
```

#   Feature	`subplot`	`subplots`	
#   -----	-----	-----	
#   Creation Style	One subplot at a time	Multiple subplots at once	
#   Returns	Single <b>Axis object</b>	<b>Figure + array of Axis objects</b>	
#   Indexing	Position via `(nrows, ncols, index)`		Access via `axes[row, col]` or `axes[i]`
#   Flexibility	Less flexible (older style)	More flexible, modern, preferred	
#   Use Case	Quick and simple plots		Complex layouts, loops, organized structure

```

```

```
🙌 In short:
```

```
Use `subplot` when you just want one or two quick plots.
```

```
***Use `subplots`** when you want a **grid of plots with more control**.
```

```

```

```
fig,axes = plt.subplots(nrows=4, ncols=3)
```

```
plt.tight_layout()
```

```
fig,axes = plt.subplots(nrows=4, ncols=3)
```

```
plt.tight_layout()
```

```
plt.plot(x,y) # by default the plot will be in last grid
```

```
Syntax to plot
```

```
axes[r,c].plot() → when grid has multiple rows & columns
```

```
axes[r].plot() → when grid has only 1 row
```

```
axes[c].plot() → when grid has only 1 column
```

```
fig,axes = plt.subplots(nrows=4, ncols=3)
```

```
plt.tight_layout()
```

```
axes[2,2].plot(x,y)
```

```
axes[2,2].set_title('Sakshi')
```

```
plt.show()
```

### ### 🧠 Step-by-step Explanation

```
1 `fig, axes = plt.subplots(nrows=4, ncols=3)`
```

# \* Creates a **Figure** (`fig`) and a **4×3** grid of subplots → total **12** subplots.

# \* `axes` is a **NumPy array** of shape **(4,3)**.

# \* `axes[0,0]` → subplot at row 1, col 1

# \* `axes[2,2]` → subplot at row 3, col 3 (since indexing starts at 0)

# \* `axes[3,2]` → subplot at row 4, col 3

# So you now have 12 empty plots available.

# PIE CHART ---> 🥧 What is a Pie Chart?

# \* A **Pie Chart** is a **circular statistical graph** that is divided into slices (like pieces of a pie).

# \* Each slice represents a **proportion (percentage)** of the whole.

# \* The **entire circle = 100%** of the data.

# For example, if you survey 100 people about their favorite fruit:

# \* 40 like Apples → 40% of the pie

```
* 30 like Mangoes → 30% of the pie
```

```
* 20 like Bananas → 20% of the pie
```

```
* 10 like Grapes → 10% of the pie
```

```
You can plot these proportions as a circle divided into 4 slices.
```

```
Syntax : plt.pie(values, labels=var)
```

```
a = list(df1['Product'])
```

```
a
```

```
plt.pie(df1['TotalSales'], labels=a)
```

```
plt.show()
```

```
Let us understand this with even more good example
```

```
b = df1.pivot_table(values='TotalSales', index='Region', aggfunc='sum')
```

```
b.reset_index(inplace=True)
```

```
b
```

```
plt.pie(b['TotalSales'], labels=b['Region'])
```

```
plt.show()
```

```
If you want to add percentage
```

```
autopct = '%1.0f%%' ---> No decimal point
```

```
autopct = '%1.2f%%' ---> Two decimal point
```

```
plt.pie(b['TotalSales'], labels=b['Region'], autopct='%1.2f%%')
plt.show()
```

```
colors ---> To change the color of each pie
```

```
plt.pie(b['TotalSales'], labels=b['Region'], autopct='%1.2f%%',
colors=['red','blue','green','yellow'])
plt.show()
```

```
Explode ---> To pull a particular slice
```

```
plt.pie(b['TotalSales'], labels=b['Region'], autopct='%1.2f%%',
colors=['red','blue','green','yellow'], explode=[0.02,0.02,0.02,0.02])
```

```
plt.show()
```

# Shadow ---> To add shadow

```
plt.pie(b['TotalSales'], labels=b['Region'], autopct='%1.2f%%',
colors=['red','blue','green','yellow'], explode=[0.1,0.02,0.02,0.02], shadow=True)
plt.show()
```

# startangle ---> startangle=90 rotates the pie so that the first slice begins from the top (12 o'clock position) instead of the default right side.

```
plt.pie(b['TotalSales'],
 labels=b['Region'],
 autopct='%1.2f%%',
 colors=['red','blue','green','yellow'],
 explode=[0.1,0.02,0.02,0.02],
 shadow=True,
 startangle=30
)
plt.show()
```

# Histogram ---> It is used to plot the count

# It accepts only a single column

# Syntax: plt.hist(data)

```
df = pd.read_csv('/content/sales_dataset.csv')
```

df

```
plt.hist(df['Product'])
```

```
plt.xticks(rotation= 90)
```

```
plt.show()
```

# 'bins' ---> Used to define the number of subdivisions (intervals) in the histogram.

# It determines how finely the data is grouped along the X-axis.

```
plt.hist(df['Product'], bins=7) # Creates a histogram of the 'Product' column with 40 bins (subdivisions).
```

```
plt.xticks(rotation=90) # Rotates the X-axis labels by 90 degrees to make them readable (especially if text is long).
```

```
plt.show() # Displays the histogram plot on the screen.
```

# 'width' ---> Used to control the width (thickness) of the bars in the histogram.

# The value ranges between 0 and 1, where:

# - Smaller values (e.g., 0.3) make the bars thinner (with gaps between them).

# - Larger values (e.g., 1) make the bars wider (bars may touch or overlap).

```
plt.hist(df['Product'], bins=40, width=0.7) # Plots a histogram of the 'Product' column
with 40 bins and sets bar width to 0.7.
```

```
plt.xticks(rotation=90) # Rotates X-axis labels by 90 degrees for better
readability.
```

```
plt.show() # Displays the histogram plot.
```

# scatter ---> Used to create a scatter plot (a graph of points) that shows the relationship

# between two numerical variables — one plotted along the X-axis and the other on the Y-axis.

# Syntax:

```
plt.scatter(x, y, color=None, marker=None, s=None)
```

# where:

# x → Values for X-axis

# y → Values for Y-axis

# color → (Optional) Defines the color of the points

# marker → (Optional) Defines the shape of the points (e.g., 'o', 's', '^')

# s → (Optional) Defines the size of the points

```
plt.scatter(df['Region'], df['TotalSales'])
```

```
plt.show()
```



```
plt.scatter(df['Region'], df['TotalSales'], color='green', marker='x', s=10)
plt.show()
```